

توثيق نمط التصميم التركيبي (Composite Pattern)

نظام إدارة الهيكل التنظيمي والرواتب

1. الواجهة المشتركة (Component Interface)

تعرف هذه الواجهة العمليات الأساسية التي يجب أن يطبقها كل من الموظف (Leaf) والقسم (Composite).

```
public interface OrganizationComponent {  
    void showDetails(); // لعرض معلومات الموظف أو هيكل القسم  
    double calculateSalary(); // لحساب راتب الموظف أو إجمالي رواتب القسم  
}
```

2. صنف الموظف (Leaf Class)

يمثل العنصر الأساسي (الورقة) في الشجرة، حيث ينفذ العمليات بشكل فردي.

```
public class Employee implements OrganizationComponent {  
    private String name;  
    private String position;  
    private double salary;  
    private int age;  
  
    public Employee(String name, String position, double salary, int age) {  
        this.name = name;  
        this.position = position;  
        this.salary = salary;  
        this.age = age;  
    }  
}
```

```

@Override
public void showDetails() {
    System.out.println("    - الموظف: " + name + " | المنصب: " + position + " | عمر: " + age);
}

@Override
public double calculateSalary() {
    return salary;
}
}

```

3. صنف القسم (Composite Class)

يمثل الحاوية التي يمكن أن تضم موظفين أو أقساماً أخرى، ويقوم بتجميع النتائج من أبنائه.

```

import java.util.ArrayList;
import java.util.List;

public class Department implements OrganizationComponent {
    private String deptName;
    private List<OrganizationComponent> components = new ArrayList<>();

    public Department(String name) {
        this.deptName = name;
    }

    public void addComponent(OrganizationComponent component) {
        components.add(component);
    }

    public int getComponentCount() {
        return components.size();
    }

    @Override
    public void showDetails() {
        System.out.println("\n--- قسم: " + deptName + " ---");
        for (OrganizationComponent component : components) {
            component.showDetails();
        }
    }
}

```

```

    }
}

@Override
public double calculateSalary() {
    double totalSalary = 0;
    for (OrganizationComponent component : components) {
        totalSalary += component.calculateSalary();
    }
    return totalSalary;
}
}

```

4. التطبيق الرئيسي (Main Application)

```

public class CompanyApp {
    public static void main(String[] args) {
        // 1. إنشاء الموظفين
        Employee emp1 = new Employee("28", 5000, "مطور جافا", "أحمد");
        Employee emp2 = new Employee("25", 6000, "مطور واجهات", "سارة");
        Employee emp3 = new Employee("30", 7000, "مصمم جرافيك", "خالد");

        // 2. إنشاء الأقسام
        Department developmentDept = new Department("التطوير");
        Department designDept = new Department("التصميم");
        Department company = new Department("الشركة الرئيسية");

        // 3. بناء الهيكل الشجري
        developmentDept.addComponent(emp1);
        developmentDept.addComponent(emp2);
        designDept.addComponent(emp3);

        company.addComponent(developmentDept);
        company.addComponent(designDept);

        // 4. عرض النتائج
        System.out.println("--- هيكل الشركة الكامل ---");
        company.showDetails();
    }
}

```

```
System.out.println("\n--- التكلفة المالية الإجمالية ---");  
System.out.println("إجمالي رواتب الشركة: " + company.calculateSalary());  
}  
}
```